Residual-Guided Hypothesis Language Induction for Scientific Reasoning with Small Language Models

Anonymous submission

Abstract

Scientific reasoning requires constructing and validating explanations from data and interaction, yet small language models remain unreliable at this task. Agentic methods augment these models with tool use, reflection, search, and executable actions. However, they usually retain failures as text or trajectory history, leaving unchanged the hypothesis language from which subsequent scientific artifacts are constructed. We introduce RG-HLI, a residual-guided hypothesis-language induction architecture that converts validation failures into reusable computational structure. Its components communicate through a shared typed state containing evidence contracts, open answer slots, executable hypotheses, validators, and residuals. Failed validations are compiled into typed residual classes, which guide the synthesis of executable operators that target recurring deficiencies in the current language. A candidate operator is promoted only when its gain on held-out tasks exceeds a complexity penalty. RG-HLI can therefore expand the executable hypothesis space available to later searches without using benchmark labels for promotion or updating the language-model weights. We evaluate RG-HLI on DiscoveryBench, NewtonBench, and UltraHorizon, which test tabular scientific discovery, symbolic law induction, and long-horizon rule discovery, respectively. Under each benchmark's native evaluator, RG-HLI improves a fixed small-model backbone while retaining the same state, validation, residual, and promotion protocol across all three settings. These results show that structured failures can support auditable growth of a model's external hypothesis language, rather than merely adding steps to its current reasoning trace.

1 Introduction

Scientific discovery asks an agent to do more than return a plausible answer. It must decide what to measure, which variables and relations are relevant, and how an attempted explanation can be falsified. DiscoveryBench exposes this problem for data and metadata (Majumder et al. 2025); NewtonBench tests interactive law induction rather than static curve fitting (Zheng et al. 2026); and UltraHorizon stresses partially observable long-horizon rule discovery (Luo et al. 2026). In all three settings, a useful answer depends on constructing the right intermediate scientific object before the target is known.

A retained DiscoveryBench artifact

QUESTION. When do axes become quantitatively most frequent in the archaeology time series?
EVIDENCE CONTRACT. Select a time window and a frequency statistic; the claim must name both.
EXECUTED EVIDENCE. Filter the axes-frequency series and return its maximum window.
RENDERED ARTIFACT. "At the end of the fourth millennium BCE, axes become quantitatively most frequent."
NATIVE CHECK. The answer matches the requested temporal facet and passes the evidence-consistency check.

Figure 1: A concrete record from the DiscoveryBench full run. The retained object contains an executable measurement and the scientific claim it licenses, rather than a free-form rationale alone.

Current inference-time scaffolds extend a model's trajectory: REACT couples reasoning with actions (Yao et al. 2023b), Reflexion and Self-Refine add verbal feedback (Shinn et al. 2023; Madaan et al. 2023), Tree-of-Thoughts and LATS search over branches (Yao et al. 2023a; Zhou et al. 2024a), and CodeAct makes executable code the action interface (Wang et al. 2024b). Their common limitation is representational. If a compact model does not express a needed coordinate system, measurement, or answer form, more sampled traces can explore a larger set of wrong programs.

We ask whether verified failures can change the *hypothesis language* available to a small proposal model. RG-HLI represents a candidate as a typed executable artifact: required evidence becomes a contract, unknown parts are typed holes, and failed checks become residual objects. The model proposes local sketches; execution closes what can be measured. Recurrent residuals propose reusable operators, but an operator is retained only after it improves held-out executable utility beyond its complexity cost. This makes the persisted object neither a transcript nor a prompt: it is a compact extension to the language from which future artifacts are constructed.

Our contributions are: (i) a shared typed state that unifies

contracts, evidence, artifacts, validators, and residuals across scientific interfaces; (ii) a residual-conditioned operator-induction protocol with held-out promotion; and (iii) full-benchmark evaluations plus a frozen language-growth audit that separates a changed hypothesis language from extra model calls.

2 Preliminaries

We write a task instance as

$$x = (q_x, D_x, \mathcal{T}_x, \mathcal{A}_x, S_x). \tag{1}$$

Here q_x is the natural-language instruction, D_x denotes observations such as tables, metadata, traces, or simulator state, $\mathcal{T}_x$ is an optional tool or environment interface, $\mathcal{A}_x$ is the admissible answer space, and $S_x : \mathcal{A}_x \to \mathbb{R}$ is the benchmark evaluator. The system observes $q_x, D_x, \mathcal{T}_x$, and the admissible output format, but does not observe the gold answer before producing its response.

A candidate answer may be a prose hypothesis, a symbolic law, a program, a transition rule, or a structured report. We use $h \in \mathcal{H}_x$ for such a scientific artifact and $R_x(h) \in \mathcal{A}_x$ for its rendered benchmark answer. The distinction is useful because an artifact may contain intermediate objects–variables, relations, measurements, code, or evidence–that are not directly shown in the final answer string.

Scientific tasks usually constrain the form of a valid artifact. We denote the task-level constraints by

$$C_x = \{s_i = (\tau_i, d_i, \nu_i)\}_{i=1}^{k}, \tag{2}$$

where τ_i is a slot type, d_i describes the required evidence or answer component, and ν_i is a local validity predicate when one is available. These constraints describe the task, not any particular solver. They specify which parts of an artifact must be grounded before the rendered answer can be considered valid.

Let $E_x(h)$ denote evidence that can be obtained from the observed interface: statistics computed from a dataset, a fitted symbolic expression, a simulator trace, a tool result, or a replay check. When an artifact violates a constraint or fails an available check, we write the discrepancy as a residual

$$\rho = (\tau_\rho, \ell_\rho, c_\rho, e_\rho, v_\rho), \tag{3}$$

where τ_ρ is the residual type, ℓ_ρ is its location in the artifact or trace, $c_\rho \in C_x$ is the relevant constraint, e_ρ is the supporting evidence, and v_ρ is the failed check or validator. This notation only records what failed; later sections describe how residuals are used by the proposed system.

A hypothesis language is a finite typed language

$$L = (\mathcal{Y}, \mathcal{O}), \tag{4}$$

where $\mathcal{Y}$ is a set of artifact and slot types, and $\mathcal{O}$ is a set of executable typed operators. For a task x, the language induces the search space $\mathcal{H}_x(L) \subseteq \mathcal{H}_x$ of artifacts that can be constructed and checked using those types and operators.

We use $\mathcal{M}$ for an abstract hypothesis state: a record of the current contract, candidate artifacts, closed slots, evidence, and residuals. At this level it is only a notational object. A language operator is any typed executable transformation of the form

$$O : \mathcal{M} \times x \to \mathcal{M}. \tag{5}$$

The concrete state representation and operator interface used by RG-HLI are defined in Section 4.

Language induction denotes a sequence of language states

$$L_0 \to L_1 \to \cdots \to L_T, \qquad L_{t+1} = L_t \cup \{o_t^\star\}, \tag{6}$$

where each added operator expands the set of artifacts that can be constructed on later tasks. This notation does not specify how $o_t^\star$ is chosen; the selection mechanism is part of the method.

The objective on each instance is to return $a = R_x(h)$ with high benchmark score $S_x(a)$. Equivalently, a scientific reasoning system must instantiate the chain

$$x \to C_x \to h \in \mathcal{H}_x(L) \to R_x(h) \to S_x(R_x(h)). \tag{7}$$

The benchmark evaluator scores only the rendered answer, but the intermediate objects determine whether a small model can construct a useful answer before seeing the gold target.

3 Related Work

Reasoning agents and executable scaffolds. Chain-of-thought, self-consistency, and least-to-most prompting make intermediate computation explicit (Wei et al. 2022; Wang et al. 2023; Zhou et al. 2023a, 2024b). ReAct, Reflexion, Self-Refine, Tree-of-Thoughts, LATS, and CodeAct add actions, feedback, tree search, tools, or code execution (Yao et al. 2023b; Shinn et al. 2023; Madaan et al. 2023; Yao et al. 2023a; Zhou et al. 2024a; Gao et al. 2023; Chen et al. 2023; Paranjape et al. 2023; Schick et al. 2023; Gou et al. 2024; Chen et al. 2024; Wang et al. 2024b). These methods improve search inside a current representation. RG-HLI is complementary: it persists a typed residual when the representation itself is inadequate, so that later search can use a changed hypothesis language rather than a longer textual trajectory.

Experience, actions, and skill libraries. ExpeL distills trajectories into textual insights, while Voyager and SkillRL retain executable or recursively evolving skill banks (Zhao et al. 2024a; Wang et al. 2024a; Xia et al. 2026). LearnAct expands an agent's action space with Python functions, and SkillWeaver and SkillFoundry compile reusable APIs or validated scientific skill packages (Zhao et al. 2024b; Zheng et al. 2025; Shen et al. 2026). These systems establish that persistent experience can improve later behavior. Our learning signal and admission unit are different: a candidate extension must be proposed from a recurrent typed validation residual, close that residual through executable state updates, and improve disjoint held-out interfaces after a complexity charge. The resulting operator is frozen before benchmark evaluation, which separates language induction from test-label adaptation.

Program induction and scientific discovery. DreamCoder learns reusable program abstractions; LILO combines language-model synthesis with symbolic compression and

documentation (Ellis et al. 2021; Grand et al. 2024). FunSearch uses externally evaluated program search to discover mathematical constructions (Romera-Paredes et al. 2024; Novikov et al. 2025). Prompt and compound-system optimizers instead search instructions, textual variables, or weight updates (Zhou et al. 2023b; Khattab et al. 2023; Fernando et al. 2024; Yuksekgonul et al. 2025; Zweiger et al. 2025). These systems learn from successful or externally scored candidates. RG-HLI instead defines the missing abstraction through failed scientific contracts: operator scope, evidence, and validators are inherited from the residual class. This connects library growth to symbolic regression and law discovery (Udrescu and Tegmark 2020; Zheng et al. 2026), invariance and information-seeking measurement (Pearl 2009; Peters, Bühlmann, and Meinshausen 2016; Rainforth et al. 2023), and automated discovery and its diagnostics (Lu et al. 2024, 2026; Chen et al. 2025; Agarwal et al. 2025).

4 RG-HLI

4.1 Overview

RG-HLI is a hypothesis-induction kernel around a small language model. The model is used to propose typed sketches, measurements, and repairs, but the state of the search is external and executable. The same loop is used for all benchmarks: compile a contract, generate artifacts, execute measurements, validate them, store residuals, and occasionally promote a residual pattern into a new operator. Operators do not pass only text to later calls; they update one typed context containing contracts, artifacts, evidence, residuals, and promoted operators.

The design is governed by three invariants. *Typed state*: every useful intermediate claim must be represented as a slot, artifact, validator, or residual rather than only as prose. *Executable closure*: whenever a slot can be identified by a computation or probe, the system closes it by execution instead of asking the model to guess it again. *Residual discipline*: a failed hypothesis can expand the operator language only when the residual type recurs and the proposed operator improves held-out tasks after a complexity penalty. These invariants are the difference between RG-HLI and a longer self-refinement transcript.

Figure 2 separates the two time scales of the method: per-task search produces a validated artifact, while development-time residual induction may change the language in which later searches operate.

4.2 The HypothesisContext

For a task x, the context is

$$\mathcal{M}_x = (C, \mathcal{H}, \Theta, \mathcal{E}, \mathcal{R}, \mathcal{O}, h^\star), \tag{8}$$

where C is the evidence contract, $\mathcal{H}$ is the pool of candidate artifacts, Θ stores closed typed holes, $\mathcal{E}$ stores executable evidence, $\mathcal{R}$ stores typed residuals, $\mathcal{O}$ is the active operator set, and $h^\star$ is the current best answer artifact. All operators read and write this object. This is the main architectural constraint: every stage must return a state update, not a private chain of thought.

Each operator $O_j \in \mathcal{O}$ has the same interface,

$$O_j(\mathcal{M}_x, x) \to \{(h, e, \rho, \Delta\mathcal{M})\}, \tag{9}$$

where h is a proposed artifact, e is executed evidence, ρ is the typed residual, and $\Delta\mathcal{M}$ is the state update. This interface is what makes the system benchmark-independent. A tabular estimand, a symbolic coordinate probe, and a transition-rule inducer differ internally, but the kernel only sees artifact, evidence, residual, and update.

Candidate scoring and typed residuals. The kernel ranks candidates by a shared energy score:

$$\begin{aligned}\text{score}(h) = \lambda_E E(h, x) + \lambda_C \,\text{cover}(h, C) \\ - \lambda_V V(h, C) - \lambda_H K_h(h).\end{aligned} \tag{10}$$

This objective is Bayesian in motivation–evidence should increase confidence, while instability and complexity should decrease it–but we do not require a calibrated likelihood or learned prior. The operational object is the benchmark-independent energy decomposition and the typed residual emitted when that score fails. The same decomposition is used across interfaces, while each operator normalizes its executable evidence to a comparable range before entering the shared ranking function. The fixed term definitions, ranges, and weights are reported in the supplement.

End-to-end trace. In a NewtonBench-style trajectory, the model does not guess the law directly: the contract asks for coordinate, exponent, constant, and residual slots; a sketch $y = c\phi(x)^p$ is executed; residual curvature activates a coordinate probe; the probe closes the coordinate, exponent, and constant slots; and the held-out validator accepts the artifact. The failed first probe changes typed state rather than merely changing the next prompt.

4.3 Operators

Contract compiler. The contract compiler maps the input interface to required slots and local validators. It does not predict the gold answer. It predicts what any valid answer must make observable: answer type, scope, variables, measurement target, relation form, and rendering constraints.

Typed-hole closure. Instead of asking the model for a complete hypothesis, RG-HLI asks for sketches with typed holes and then closes the holes by small executable probes whenever possible. For a tabular question, a hole may be an outcome role or a window operator. For a law-discovery task, it may be a coordinate chart or exponent. For a sequence task, it may be a state variable or transition clause. Closing a hole adds an assignment to Θ and reduces the search space for later holes.

Executable estimands and coordinate probes. The estimand operator creates statistical objects for tabular discovery: group contrasts, interaction effects, coefficient shifts, multi-item proportions, and time-window measurements. The coordinate-probe operator creates compact transformations for laws and rules: log-log fits, affine and polynomial lifts, ratios, finite differences, thresholds, and simple state-transition clauses. Both operators produce evidence objects rather than only prose rationales.

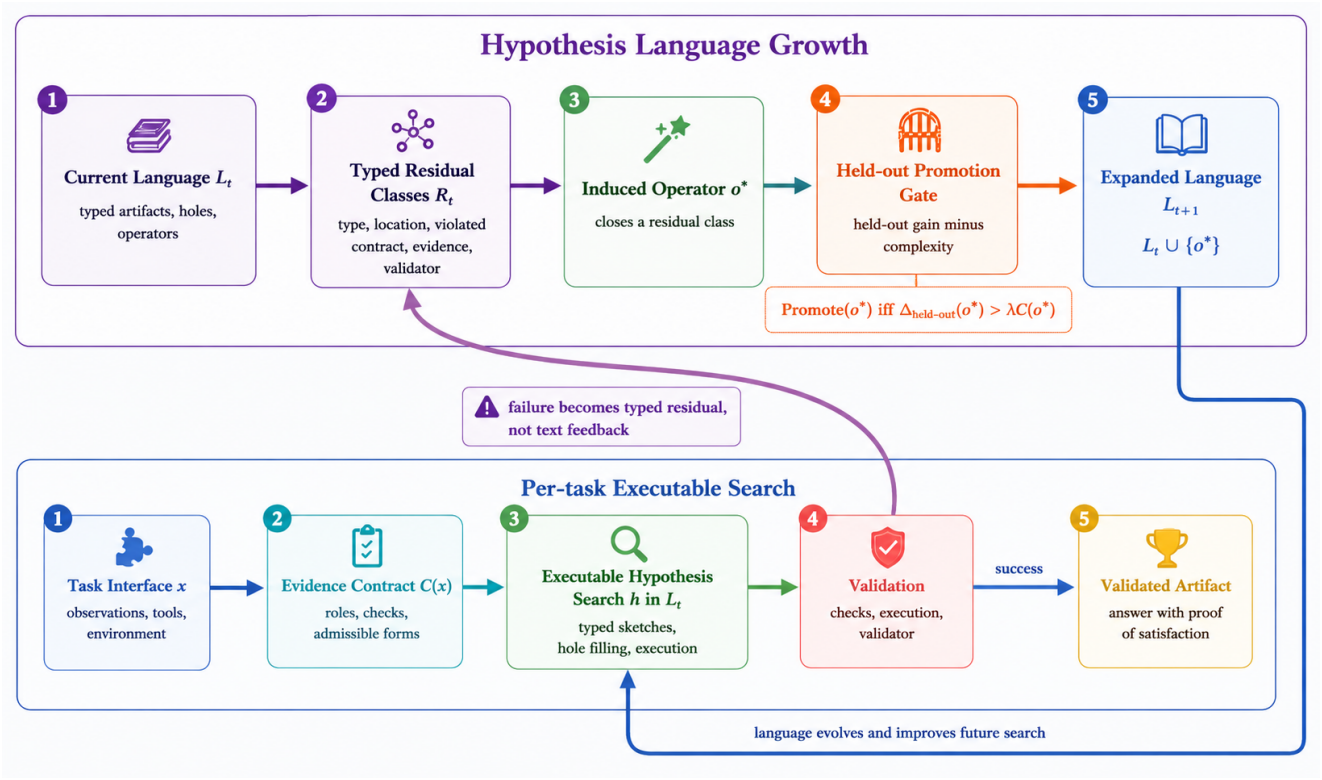

Figure 2: Hypothesis-language growth in RG-HLI. The lower loop constructs and validates an artifact for the current task. A failed validation becomes a typed residual rather than textual feedback. The upper loop admits a reusable operator only when it improves held-out utility beyond its complexity cost, thereby changing the language available to later tasks.

Metamorphic verifier. The verifier checks whether the artifact answers the requested question under transformations that should preserve the object: renaming irrelevant columns, holding answer form fixed, changing display order, perturbing nuisance values, or replaying held-out traces. These checks are label-free and therefore available before benchmark evaluation.

4.4 Induction Algorithms

The procedure runs at two time scales. At benchmark time, the language $L^\star$ is frozen. The system compiles C_x, initializes $\mathcal{M}_x$, repeatedly selects admissible operators, executes their probes, writes artifacts/evidence/residuals back into the context, updates $h^\star$ by the shared score, and finally renders $R_x(h^\star)$. Residuals from reported test rows may be logged for analysis, but they cannot change $L^\star$ for later reported rows. At development time, residuals are canonicalized and grouped by residual type, location class, contract role, and evidence geometry. The reported mechanism audit uses deterministic signatures and requires at least two compatible source residuals. Each recurrent class may propose a candidate operator through the same small-model code-and-sketch interface used for task solving. The operator is discarded unless it passes static type checks, sandbox execution, validators, and the held-out promotion rule below. Full pseudocode is provided in the supplement.

Language-growth protocol. The promotion step has two modes. During development, residuals may induce candidate operators, which are promoted only if they improve a held-out development set after the complexity penalty in Eq. (11). For the headline benchmark runs, the resulting operator language $L^\star$ is frozen before the reported evaluation. The benchmark score is never used as the promotion signal for examples later reported as test results. Online adaptation is therefore limited to per-task hypothesis search in the frozen language, not to changing the language using future test labels.

Running example. Figure 3 traces the first accepted NewtonBench extension in the frozen audit. It illustrates the intended separation: repeated failures identify an operator class; held-out utility decides promotion; and a disjoint task benefits from the changed hypothesis language.

4.5 Residual-Conditioned Operator Promotion

The residual-extension step is not arbitrary code growth. A residual cluster $R = \{\rho_1, \ldots, \rho_m\}$ can propose an operator only if it identifies a repeatable missing transformation. Formally, a candidate operator O' is promoted when

$$G(O') = \mathbb{E}_{x \sim \mathcal{D}_{held}}[U_x(h^\star_{O'}(x)) - U_x(h^\star(x))] - \beta K_o(O') > 0, \tag{11}$$

where U_x is a development-time executable validation utility, not the reported benchmark evaluator S_x. Source and pro-

A retained NewtonBench language-growth trace	
DEVELOPMENT EVIDENCE	Gravity and Coulomb modules both leave the same typed residual: a positive structured relation cannot be closed in the current coordinate system.
RESIDUAL-CONDITIONED PROPOSAL	Propose a log-coordinate operator that fits exponents by execution: $\log y = a + \sum_i p_i \log x_i$. The operator is a typed transform with declared inputs, output form, and validators.
HELD-OUT PROMOTION	Evaluate the transform on disjoint promotion modules and subtract its program-complexity cost. The utility is positive, so the operator is added to L_{t+1} rather than stored as a solved example.
DISJOINT TRANSFER	On the unseen Hooke module, the extended language exposes the force–displacement family and lowers executable loss from 1.000 to 0.206, with no new operator induction.

Figure 3: An audited, concrete language-growth step. The persistent object is a small executable coordinate transform, not a Gravity, Coulomb, or Hooke answer cached for reuse.

motion tasks are disjoint, and at least one promotion interface must improve. All promotable transforms are serialized into a common executable intermediate representation, with $K_o(O') = \log(1 + n_{\mathrm{AST}}(O'))$. We fix $\beta = 10^{-3}$ before the reported trajectory. This rule limits uncontrolled library growth: an operator must improve non-source tasks under the same validation interface while paying a complexity penalty.

4.6 Why This Helps Small Models

RG-HLI changes the role of the language model. The model does not need to hold the entire long-horizon proof in context or invent the final hypothesis in one sample. It repeatedly proposes local typed objects whose consequences can be executed. Each closed hole constrains later holes, each verifier rejects an invalid answer form before final scoring, and each residual becomes a structured training signal for the outer loop. The intended gain is therefore not more verbal reasoning, but a better external search geometry for capacity-limited models.

5 Experimental Evaluation

5.1 Benchmarks and protocol

We evaluate on three benchmark families that stress different scientific interfaces while sharing the same central difficulty: the agent must construct a hypothesis object before it can answer. DiscoveryBench provides real datasets, metadata, and natural-language discovery goals; the final answer is scored by Hypothesis Matching Score (HMS) and a consistency-HMS variant. NewtonBench provides interactive scientific-law discovery tasks in shifted physical worlds; we report symbolic accuracy over all tasks (SA-all) and over non-abstained answers (SA-answered). UltraHorizon provides long-horizon partially observable rule-induction environments; we report the reproduced paper-style environment score.

The headline system uses a GPT-4o-mini proposal core. We use one shared hypothesis-induction stack: contract compilation, typed-hole closure, estimand synthesis, coordinate probing, metamorphic validation, and residual-conditioned promotion. Benchmark adapters expose observations, validators, and final renderers; interface-specific operator families expose measurable objects appropriate to each environment, but they all write the same HYPOTHESISCONTEXT and obey the same residual and promotion interface. Because the artifact types differ, evaluation uses a separately frozen $L^\star_{\mathrm{DB}}$, $L^\star_{\mathrm{NB}}$, and $L^\star_{\mathrm{UH}}$, rather than claiming one identical operator library. We use "small" operationally to denote the lower-cost GPT-4o-mini proposal tier, not an undisclosed parameter-count threshold. We exclude diagnostic ceiling runs that used task-family measurement bootstraps or hand-engineered rendering fixes.

Proposal core	DB HMS	NB SA-all	UH score
GPT-4o-mini	28.70	29.01	**75.36**
Gemini 2.5 Flash Lite	**29.14**	**29.32**	65.42
Qwen3-30B-A3B-Instruct	28.58	**29.32**	24.64

Table 1: Complete proposal-core substitutions. Each column fixes the task set, executor, and evaluator while changing only the proposal endpoint. DB uses the matched 239-task protocol; NB uses the 324-task diagnostic kernel; UH uses the 96-episode controller-enabled protocol. Scores are comparable down columns, not across metrics or with differently configured headline rows.

Complete row counts, evaluator configurations, confidence intervals, and the frozen-language protocol are reported in the supplement. In particular, the UltraHorizon result uses the strict configuration, with fallback commits, environment hints, terminal controllers, and measurement bootstraps disabled.

5.2 Main comparisons

Table 2 reports the complete benchmark runs together with published original-paper references and our reproduced controls. DiscoveryBench and NewtonBench include complete GPT-4o-mini REACT/CODEACT-style controls under the same task set and evaluator as RG-HLI. The strict UltraHorizon control instead isolates the shared kernel from its optional components over all 96 episodes. All RG-HLI operator languages are frozen before reported evaluation. The canonical NewtonBench result therefore evaluates the typed executable kernel, while development-time language growth is tested separately below. Table 1 additionally shows that the executable architecture remains operational with Gemini and Qwen proposal cores.

Method	Core	Evidence	Native score
DiscoveryBench: HMS (Majumder et al. 2025)			
CodeGen	GPT-4-preview	Published	16.3
ReAct	GPT-4-preview	Published	15.6
Reflexion (oracle)	GPT-4o	Published	24.5
Direct	GPT-4o-mini	Reproduced	9.54
ReAct	GPT-4o-mini	Reproduced	11.68
CodeAct	GPT-4o-mini	Reproduced	12.31
RG-HLI	GPT-4o-mini	Ours	**28.70** [23.24, 34.28]
NewtonBench: SA-all (%) (Zheng et al. 2026)			
Vanilla agent	GPT-5	Published	75.9
Vanilla agent	o4-mini	Published	47.8
Interactive ReAct-style agent	GPT-4o-mini	Reproduced	0.00
CodeAct-style agent	GPT-4o-mini	Reproduced	0.62
RG-HLI	GPT-4o-mini	Ours	**49.38** [44.14, 54.94]
UltraHorizon: environment score (Luo et al. 2026)			
Best free-step LLM	Gemini-2.5-Pro	Published	14.33
Human participants	human	Published	26.52
Strict all-off agent	GPT-4o-mini	Reproduced	0.00
RG-HLI strict	GPT-4o-mini	Ours	**51.04** [44.79, 56.88]

Table 2: Complete benchmark comparisons. Amber rows are original-paper reports, blue rows are our complete reproduced controls, and green rows are our complete RG-HLI evaluations. Scores use each benchmark's native metric and are comparable only within a block. The DiscoveryBench oracle row is contextual; intervals are 95% nonparametric bootstrap intervals.

5.3 Mechanism evidence

To isolate the mechanism, we freeze observations, parameter fitting, and test-time calls in a NewtonBench diagnostic. Residual-conditioned selection reduces held-out executable loss by 0.171 ± 0.012 against a type-compatible additive control across 12 paired evaluations (four interfaces and three seeds). Figure 4 shows the complete 324-task profile and disjoint promotion evidence; matched controls pass the same parser and judge. The supplement gives prompts, category audit, and traces.

5.4 What the full runs show

The full runs test one typed-state kernel across three scientific interfaces. DiscoveryBench reaches 28.70 HMS, versus 9.54–12.31 for matched Direct, ReAct, and CodeAct controls. NewtonBench reaches 49.38% SA-all at 74.1% coverage; matched interactive and code-action controls reach 0.00% and 0.62%. On UltraHorizon, the strict configuration preserves an induced program across a fixed 50-step horizon and reaches 51.04. The state schema and promotion interface are shared, while each benchmark freezes a language appropriate to its artifact type. Separate frozen-transfer and multi-step audits test language growth; the supplement provides residual taxonomies, provenance, and traces.

These experiments separate system-level utility from the mechanism claim. The complete runs use native evaluators, paired controls fix the task set, backbone, parser, and evaluator, and the disjoint audit tests transfer across interfaces. Because admission requires held-out loss reduction after a complexity charge, the 0.171 ± 0.012 advantage and the accepted–rejected updates in Figure 4 and the audited trace below specifically test language growth.

5.5 Proposal-core portability

Table 1 replaces the proposal endpoint without changing the typed state, executable operators, task order, or evaluator. Across complete runs, DiscoveryBench varies by 0.56 HMS and the frozen NewtonBench diagnostic by 0.31 SA-all. UltraHorizon shows stronger endpoint–environment interaction, but all cores complete the same 96 episodes without model-specific operators or prompts. These substitutions test interface portability and are not pooled with the strict headline protocols.

One audited language update. Gravity and Coulomb interfaces produced a recurrent positive, log-structured residual class. The residual-conditioned proposal was a positive-monomial lattice:

$$L_0 \xrightarrow[0.6055-0.0067=+0.5988]{\text{held-out promotion}} L_1 = L_0 \cup \{o_{\text{mono}}\}.$$

After freezing L_1, the operator reduced executable loss by 0.5871 on disjoint transfer interfaces. A repeated copy later had net gain -0.0067 and was rejected. Admission used disjoint executable loss plus the declared complexity charge; benchmark scores and reported-test residuals were unavailable to the gate.

Update	Disjoint evidence	Outcome
$L_0 \to L_1$	gravity, Coulomb → magnetic, Fourier	$+0.5988$
frozen L_1	decay, Hooke, heat-transfer	$+0.5871$
$L_1 \to L_2$	underdamped, Malus → sound-speed	$+0.0082$
frozen L_2	Bose–Einstein distribution	$+0.0996$
duplicate	repeated monomial source hash	-0.0067

The admitted operator changes the executable search space before transfer; recurrence alone does not guarantee promotion, and the language is frozen before reported rows.

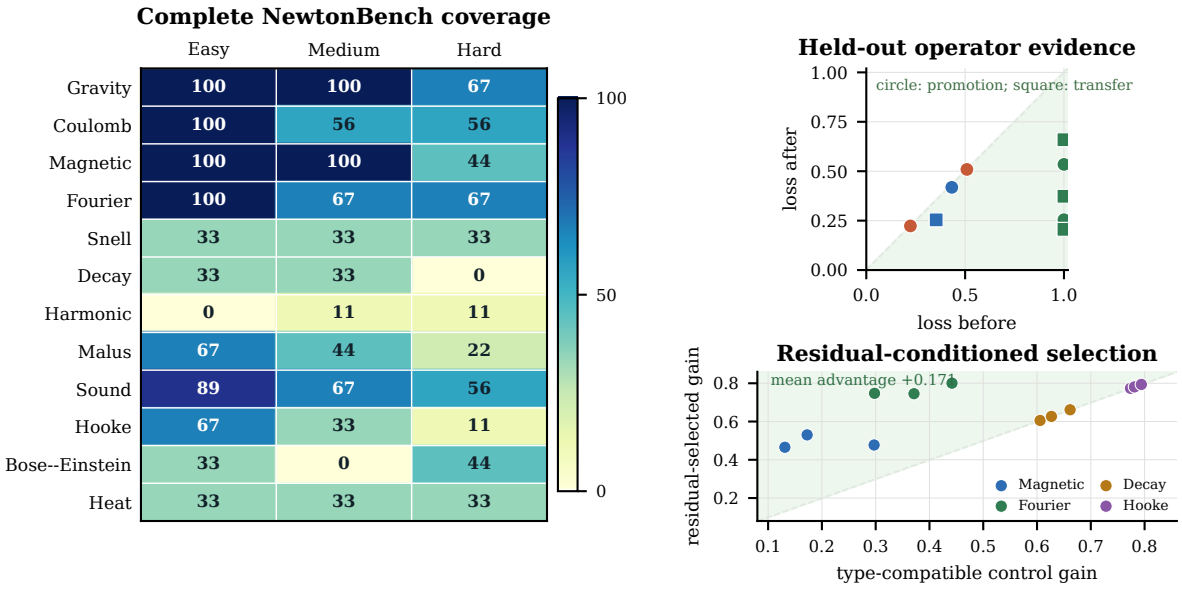

Figure 4: Complete NewtonBench coverage and the frozen language-growth audit. Left: exact symbolic accuracy over all 324 configurations. Upper right: executable loss before and after accepted operators on held-out and disjoint interfaces. Lower right: 12 paired evaluations compare residual-conditioned selection with a type-compatible control under identical fitting and test-time compute.

6 Limitations

RG-HLI assumes that scientific intermediates admit executable contracts, typed holes, validators, renderers, or residual-induced operators. The current implementation covers three artifact classes and fixed energy weights; richer semantic renderers and learned held-out calibration remain open. Because the benchmarks reward semantic agreement, symbolic equivalence, and long-horizon behavior, respectively, we report native metrics rather than one aggregate. The shared object is the typed-state and promotion protocol, not an identical operator library. The strict UltraHorizon result succeeds on Grid and Seq but does not solve Bio or provide a matched external-agent control.

RG-HLI promotes only operators that can be serialized, executed, and tested on disjoint interfaces. It makes representational growth inspectable rather than assuming that every scientific concept has a short program. When evidence or a validator is unavailable, the system retains an unresolved residual instead of promoting a plausible verbal repair.

Reported-test residuals, benchmark scores, and gold laws are unavailable to the promotion gate and cannot rewrite the frozen language.

Matched comparisons fix the backbone, task set, parser, and evaluator, but structured execution uses more calls than direct prompting. The supplement reports call records and provenance; the claim is utility at the stated budget, not a compute-matched Pareto frontier. "Small" denotes the provider's compact, low-cost tier rather than an undisclosed parameter threshold.

NewtonBench SA-all counts abstentions as errors; coverage is 74.1%, and the supplement reports SA-answered separately. The UltraHorizon headline excludes the public terminal controller.

The proposal-core substitutions are complete benchmark-sized runs, but they are not a parameter-scaling study: provider sizes and training mixtures are not consistently disclosed. Their supported claim is architectural portability. The same serialized state, executor, validators, and renderer operate when the proposal endpoint changes, while endpoint-dependent search quality can still vary by environment. Open-weight scaling and compute-matched Pareto comparisons are useful next tests rather than assumptions required by the present result.

7 Conclusion

RG-HLI turns validation failures into typed operators that can alter the hypotheses available to later searches. Complete native-evaluator runs establish utility across tabular discovery, symbolic laws, and long-horizon rules; matched controls and proposal-core substitutions separate the architecture from one prompt or provider. Most importantly, the accepted–rejected operator trajectory and disjoint frozen transfer test the paper's central mechanism: the system improves by changing an executable hypothesis language, not merely by extending the current reasoning trace.

Taken together, the results establish task utility, mechanism evidence, and proposal-core portability. Recurrent residuals identify missing computational structure, while disjoint promotion prevents a solved example from being mistaken for a reusable operator. RG-HLI thus provides a concrete route from an observed failure to an inspectable language update whose effect can be measured on later tasks.

References

Agarwal, D.; Majumder, B. P.; Adamson, R.; Chakravorty, M.; Gavireddy, S. R.; Parashar, A.; Surana, H.; Dalvi Mishra, B.; McCallum, A.; Sabharwal, A.; and Clark, P. 2025. AutoDiscovery: Open-Ended Scientific Discovery via Bayesian Surprise. In *Advances in Neural Information Processing Systems*.

Chen, T.; Lin, B.; Anumasa, S.; Shah, V.; Yuan, Z.; Zou, Q.; Goyal, A.; and Liu, D. 2025. Auto-Discovery-Bench: Diagnosing Structured State Tracking in Oracle-Guided Discovery. *arXiv preprint arXiv:2502.15224*.

Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*.

Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *International Conference on Learning Representations*.

Ellis, K.; Wong, C.; Nye, M.; Sablé-Meyer, M.; Morales, L.; Hewitt, L.; Cary, L.; Solar-Lezama, A.; and Tenenbaum, J. B. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*.

Fernando, C.; Banarse, D. S.; Michalewski, H.; Osindero, S.; and Rocktäschel, T. 2024. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In *International Conference on Machine Learning*.

Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2023. PAL: Program-Aided Language Models. In *International Conference on Machine Learning*.

Gou, Z.; Shao, Z.; Gong, Y.; Shen, Y.; Yang, Y.; Duan, N.; and Chen, W. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*.

Grand, G.; Wong, L.; Bowers, M.; Olausson, T. X.; Liu, M.; Tenenbaum, J. B.; and Andreas, J. 2024. LILO: Learning Interpretable Libraries by Compressing and Documenting Code. In *International Conference on Learning Representations*.

Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; Joshi, T. T.; Moazam, H.; Miller, H.; Zaharia, M.; and Potts, C. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714*.

Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.

Lu, C.; Lu, C.; Lange, R. T.; Yamada, Y.; Hu, S.; Foerster, J.; Ha, D.; and Clune, J. 2026. Towards End-to-End Automation of AI Research. *Nature*, 651(8107): 914–919.

Luo, H.; Zhang, H.; Zhang, X.; Wang, H.; Qin, Z.; Lu, W.; Ma, G.; He, H.; Xie, Y.; Zhou, Q.; Hu, Z.; Mi, H.; Wang, Y.; Tan, N.; Chen, H.; Fung, Y.; Yuan, C.; and Shen, L. 2026. UltraHorizon: Benchmarking LLM-Agent Capabilities in Ultra Long-Horizon Scenarios. In *International Conference on Machine Learning*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*.

Majumder, B. P.; Surana, H.; Agarwal, D.; Dalvi Mishra, B.; Meena, A.; Prakhar, A.; Vora, T.; Khot, T.; Sabharwal, A.; and Clark, P. 2025. DiscoveryBench: Towards Data-Driven Discovery with Large Language Models. In *International Conference on Learning Representations*.

Novikov, A.; Vũ, N.; Eisenberger, M.; Dupont, E.; Huang, P.-S.; Wagner, A. Z.; Shirobokov, S.; Kozlovskii, B.; Ruiz, F. J. R.; Mehrabian, A.; Kumar, M. P.; See, A.; Chaudhuri, S.; Holland, G.; Davies, A.; Nowozin, S.; Kohli, P.; and Balog, M. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *arXiv preprint arXiv:2506.13131*.

Paranjape, B.; Lundberg, S.; Singh, S.; Hajishirzi, H.; Zettlemoyer, L.; and Ribeiro, M. T. 2023. ART: Automatic Multi-Step Reasoning and Tool-Use for Large Language Models. *arXiv preprint arXiv:2303.09014*.

Pearl, J. 2009. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition.

Peters, J.; Bühlmann, P.; and Meinshausen, N. 2016. Causal Inference by Using Invariant Prediction: Identification and Confidence Intervals. *Journal of the Royal Statistical Society: Series B*, 78(5): 947–1012.

Rainforth, T.; Foster, A.; Ivanova, D. R.; and Bickford Smith, F. 2023. Modern Bayesian Experimental Design. *arXiv preprint arXiv:2302.14545*.

Romera-Paredes, B.; Barekatain, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J. R.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; Kohli, P.; and Fawzi, A. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature*, 625(7995): 468–475.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.

Shen, S.; Cheng, W.; Ma, M.; Turcan, A.; Zhang, M. J.; and Ma, J. 2026. SKILLFOUNDRY: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. *arXiv preprint arXiv:2604.03964*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*.

Udrescu, S.-M.; and Tegmark, M. 2020. AI Feynman: A Physics-Inspired Method for Symbolic Regression. *Science Advances*, 6(16): eaay2631.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024a. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*.

Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; and Ji, H. 2024b. Executable Code Actions Elicit Better LLM Agents. In *International Conference on Machine Learning*.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234*.

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Yuksekgonul, M.; Bianchi, F.; Boen, J.; Liu, S.; Lu, P.; Huang, Z.; Guestrin, C.; and Zou, J. 2025. Optimizing Generative AI by Backpropagating Language Model Feedback. *Nature*, 639(8055): 609–616.

Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024a. ExpeL: LLM Agents Are Experiential Learners. In *AAAI Conference on Artificial Intelligence*.

Zhao, H.; Ma, C.; Wang, G.; Su, J.; Kong, L.; Xu, J.; Deng, Z.-H.; and Yang, H. 2024b. Empowering Large Language Model Agents through Action Learning. In *Conference on Language Modeling*.

Zheng, B.; Fatemi, M. Y.; Jin, X.; Wang, Z. Z.; Gandhi, A.; Song, Y.; Gu, Y.; Srinivasa, J.; Liu, G.; Neubig, G.; and Su, Y. 2025. SkillWeaver: Web Agents Can Self-Improve by Discovering and Honing Skills. *arXiv preprint arXiv:2504.07079*.

Zheng, T.; Tam, K. K.-W.; Nguyen, N. H.-N. K.; Xu, B.; Wang, Z.; Cheng, J.; Tsang, H. T.; Wang, W.; Bai, J.; Fang, T.; Song, Y.; Wong, G. Y.; and See, S. 2026. NewtonBench: Benchmarking Generalizable Scientific Law Discovery in LLM Agents. In *International Conference on Learning Representations*.

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2024a. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *International Conference on Machine Learning*.

Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q. V.; and Chi, E. H. 2023a. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *International Conference on Learning Representations*.

Zhou, P.; Pujara, J.; Ren, X.; Chen, X.; Cheng, H.-T.; Le, Q. V.; Chi, E. H.; Zhou, D.; Mishra, S.; and Zheng, H. S. 2024b. SELF-DISCOVER: Large Language Models Self-Compose Reasoning Structures. In *Advances in Neural Information Processing Systems*.

Zhou, Y.; Muresanu, A. I.; Han, Z.; Paster, K.; Pitis, S.; Chan, H.; and Ba, J. 2023b. Large Language Models Are Human-Level Prompt Engineers. In *International Conference on Learning Representations*.

Zweiger, A.; Pari, J.; Guo, H.; Akyürek, E.; Kim, Y.; and Agrawal, P. 2025. Self-Adapting Language Models. *arXiv preprint arXiv:2506.10943*.